Tottie Master Documentation

NKNGO AI Labs

October 26, 2026

Contents

I Documentation Standards 1

1 Agent Documentation Standards 2
- 1.1 Core Documentation Mandate . . . 2
  - 1.1.1 Rule 1: The LaTeX Mandate . . . 2
  - 1.1.2 Rule 2: The Unified Root (`master.tex`) . . . 2
  - 1.1.3 Rule 3: Post-Implementation Documentation Mandate . . . 2
- 1.2 Process Flow Mapping Standards (The Cookbook Format) . . . 2
  - 1.2.1 Structuring the `process_flows/` Directory . . . 3
  - 1.2.2 The Process Cookbook Format . . . 3
- 1.3 Local Orchestration and Observability Standards (`orch-cli`) . . . 3

II Vision 5

2 Why Tottie Exists 6
- 2.1 The Problem: Agents Have Amnesia . . . 6
- 2.2 The Thesis: Intelligence at Write Time . . . 6
- 2.3 Positioning Against the 2026 Field . . . 6
- 2.4 Why It Matters (the human framing) . . . 6

III Business and Hackathon 8

3 Hackathon Rules, Dates, and Judging 9
- 3.1 Problem Statements . . . 9
- 3.2 Mandatory Meta Tooling . . . 9
- 3.3 Dates (all times Eastern) . . . 9
- 3.4 Hard Rules . . . 9
- 3.5 Sample Anti-Projects (things to avoid resembling) . . . 10
  - 3.5.1 How Tottie differs from the banned “Basic RAG Application” . . . 10
- 3.6 Judging Rubric (equally weighted, 0–10) . . . 10
- 3.7 Prizes We Target . . . 10

4 Hackathon Compliance Ledger 11
- 4.1 Statement of Original Contribution . . . 11
- 4.2 Feature $\rightarrow$ Commit Ledger . . . 11
- 4.3 Reuse Disclosure . . . 11

IV System Architecture 12

5 System Architecture 13
- 5.1 Overview . . . 13

5.2 The Four Minds . . . 13
5.3 The Tier Model . . . 14
5.4 Write Path (summary) . . . 14
5.5 Recall Path (summary) . . . 14
5.6 Reuse Map (patterns, not files) . . . 14

6 Data Model and Storage **15**
6.1 Storage Decision (documented exception) . . . 15
6.2 Schema . . . 15
6.3 Indexes . . . 16
6.4 Temporal Correctness . . . 16
6.5 Owner Scoping . . . 16
6.6 Embeddings . . . 16

7 Model Providers and the Intelligence Layer **17**
7.1 Provider Abstraction . . . 17
7.2 Component Model Choices . . . 17
7.3 Cost Control . . . 17
7.4 Optional: MCP Sampling (flagged, off by default) . . . 17

8 MCP Surface **19**
8.1 Tools . . . 19
8.2 Resources . . . 19
8.3 Prompt . . . 19
8.4 Dual-Path Recall (the design answer) . . . 19

9 Tottie UI and the Live Memory Panel **20**
9.1 Layout . . . 20
9.2 The Panel Is a Live Stream of `memory_events` . . . 20
9.3 Tottie Avatar — Drop-In Sprite (frame sets per state) . . . 20
9.4 Ablation Toggle . . . 21

V Process Flows **22**

10 Process Flow: The Write Path **23**
10.1 The User Says Something Durable . . . 23
10.2 The Observer Decides . . . 23
10.3 The Scribe Structures and Embeds . . . 23
10.4 Reconcile, Route, and Persist . . . 24
10.5 Why This Order Matters . . . 24

11 Process Flow: The Recall Path **25**
11.1 A New Turn Arrives . . . 25
11.2 The Identity Slice Is Always Included . . . 25
11.3 Hybrid Search . . . 25
11.4 The Procedural Slice . . . 25
11.5 The Curator Builds the Context Card . . . 26
11.6 The Deliberate Probe . . . 26
11.7 Ablation . . . 26

12 Process Flow: Reconcile and Temporal Correctness **27**
12.1 The Decision Table . . . 27
12.2 Worked Example: “I’m vegetarian now” . . . 27
12.3 Why This Is Hard and Why It Wins . . . 28

13 Process Flow: The Demo Click-Path **29**
13.1 Beat 1 — Cold Open (Amnesia) . . . 29
13.2 Beat 2 — Tottie Wakes (Identity Pinning) . . . 29
13.3 Beat 3 — Write-Time Magic (Supersede) . . . 29
13.4 Beat 4 — Three Bodies (Cross-Agent Recall) . . . 29
13.5 Beat 5 — Autosave a Mistake . . . 29
13.6 Beat 6 — Restart + Ablation . . . 30
13.7 Beat 7 — The Joke and the Pitch . . . 30

VI Planning **31**

14 Roadmap **32**
14.1 Scope Tiers (9 days, solo, alongside work and study) . . . 32
14.2 Phase Schedule . . . 32
14.3 Gating . . . 32
14.4 Mandates Honoured During the Build . . . 33

15 Master Todo **34**
15.1 Phase 0 — Scaffold . . . 34
15.2 Phase 1 — Documentation . . . 34
15.3 Phase 2 — Deck . . . 34
15.4 Phase 3 — Core Engine . . . 34
15.5 Phase 4 — MCP . . . 34
15.6 Phase 5 — Round-Trip . . . 34
15.7 Phase 6 — UI . . . 35
15.8 Phase 7 — Reconcile / Tags / Identity / Lessons . . . 35
15.9 Phase 8 — Integration + Eval . . . 35
15.10 Phase 9 — Film . . . 35
15.11 Phase 10 — Submit . . . 35

16 Demo Script (3 minutes, screen capture + captions + music) **36**
16.1 Shot List . . . 36
16.2 Narration Captions (verbatim) . . . 36
16.3 Music and Pacing . . . 37

17 Plan Audit (Numbered Outline and Method) **38**
17.1 Audit Method . . . 38
17.2 Independent-Review Questions (for third-party judges) . . . 38
17.3 The Numbered Outline . . . 38
17.4 Audit Ledger . . . 39

Part I

Documentation Standards

Chapter 1

Agent Documentation Standards

1.1 Core Documentation Mandate

This document serves as the absolute source of truth for any Autonomous AI Agent (including Gemini, Loid3, or other models) operating across the ENTIRE NKNGO Portals Ecosystem (including NKNGO-LANG, LearnersQuizNamibia, Pattern, and all future projects). When asked to write, update, or analyze documentation, you MUST adhere to the following universal standardization rules without exception.

1.1.1 Rule 1: The LaTeX Mandate

- **No Markdown Allowed:** All architectural, system, and process flow documentation MUST be written in LaTeX(`.tex`) format.
- **Exceptions:** The **only** Markdown files permitted in the root directory are `README.md` (for GitHub previews) and `GEMINI.md` (the immutable core system prompt for agents).
- **Deletion Clause:** If an agent generates or discovers standalone `.md` files containing system documentation, they must convert them to `.tex`, move them into the `docs/` directory, and permanently delete the `.md` file.

1.1.2 Rule 2: The Unified Root (`master.tex`)

- All documentation lives inside the `/docs` directory.
- You must never create isolated `.tex` files. Every new file must be logically categorized into a subdirectory (e.g., `process_flows/`, `business/`, `system/`) and formally imported via an `\input{}` command inside `docs/master.tex`.

1.1.3 Rule 3: Post-Implementation Documentation Mandate

- Whenever an agent implements a new feature, refactors a system, or writes new code, they MUST immediately document the flows and architecture in the appropriate LaTeX files. Do not wait for the user to ask for documentation after coding; it is an automatic, mandatory requirement.

1.2 Process Flow Mapping Standards (The Cookbook Format)

When mapping flows, DO NOT generate massive dumps of raw code, line-by-line syntax, or meaningless high-level abstractions. Documentation must represent the **Human/System Process Journey**. We are writing cookbooks that map how a user gets from A to B so we can understand the architecture before coding.

1.2.1 Structuring the `process_flows/` Directory

Flows must be organized by logical **Domains**, not arbitrarily by file name. Example structure:
- `docs/process_flows/tasks_and_projects.tex`
- `docs/process_flows/finance_and_ledgers.tex`
- `docs/process_flows/social_and_friends.tex`
- `docs/process_flows/system_and_sync.tex`

1.2.2 The Process Cookbook Format

Every feature within a domain must be broken down into explicit, human-readable steps that explain the complete logic loop. Use standard LaTeXlists and sections.

CRITICAL ANTI-ABSTRACTION MANDATE: Do **NOT** write code-focused abstracts for the UI/Action layers (e.g., *"The getChildren() method of the NoteProvider fires"*). You **MUST** describe the physical human action (e.g., *"The user expands the Notes folder in the sidebar"*).

BRUTAL UI GRANULARITY MANDATE: Never map flows at a surface level (e.g., "The user creates an account" or "The user logs an expense"). You **MUST** map the exact, granular physical UI elements the user interacts with. This means explicitly documenting the specific horizontal scrolling chip selectors, the exact preset labels (e.g., FNB, Bank Windhoek), the exact toggles for recurring actions (e.g., Weekly vs Monthly), and the specific text inputs (e.g., "Quantity" and "Cost Basis"). If the UI has a specific switch or dropdown, the documentation must explicitly state it. Keep mapping and "adding on until it's really all there".

Required elements for EVERY mapped feature:
1. **Trigger:** How does the process start physically? (e.g., "User swipes left on a Todo in the Inbox")
2. **Action / UI State:** What does the user see and interact with? (e.g., "Action bubble menu appears, user selects 'Bump to Tomorrow'")
3. **System Logic:** What is the intended behavior and calculation behind the scenes? (e.g., "System recalculates the due date to tomorrow at 00:00:00 and removes the item from the Today deck")
4. **Data Flow & Mutators:** Where does the data go? (e.g., "Updates local WatermelonDB SQLite -> Queues offline mutation in AsyncStorage -> Synchronizes via POST /api/sync to Drizzle ORM when online")

CRITICAL CROSS-SCREEN MANDATE: Flows are highly dynamic and rarely siloed to a single screen. You **MUST** explicitly map deep inter-screen connections (e.g., "Notes flow pushes parameters to FinanceScreen to auto-add expenses" or "Project Focus forces the global Calendar to filter by Project ID"). Never stop at the boundary of a single screen if the user journey naturally bridges into another domain.

Failure to adhere to the Domain Structure, the Brutal UI Granularity Mandate, and the Cookbook Format will result in bloated, unreadable documentation and constitutes a complete failure of the agent's architectural mandate.

1.3 Local Orchestration and Observability Standards (`orch-cli`)

All projects across the NKNGO Portals ecosystem are orchestrated locally via `orch-cli` located in `launcher/orch-cli`. Autonomous AI Agents MUST NEVER guess whether a server, bundler, or service is functioning. Agents must operate with absolute physical observability:
1. **Status Inspection:** Run `orch status` (or `orch status --json`) to verify the active daemon, port, PID, profile, and all registered child processes.
2. **Error Isolation:** Run `orch logs -e` to isolate errors, unhandled exceptions, and 500s across all frameworks without wading through routine log clutter.
3. **Browser Telemetry:** Run `orch logs BROWSER -n 50` to inspect live client-side Chrome console errors, React component crashes, and failed network requests.

4. **Service-Specific Logs:** Run `orch logs <SERVICE> -n 50` (e.g., `orch logs MOBILE-WEB` or `orch logs API`) to audit backend and bundler output.
5. **Hot Reloading:** Run `orch restart` to signal the active daemon to reload all child services upon applying code fixes.
6. **Profile Switching:** Launch targeted multi-service environments with `orch run -p "<profile>"` (e.g., `orch run -p "Pattern + Ask AI (Test Local)"`).

Part II

Vision

Chapter 2

Why Tottie Exists

2.1 The Problem: Agents Have Amnesia

Every agent on every platform wakes up stateless. It has no history, no user context, and no memory of yesterday. A year of conversations later, it still does not know who you are, what you build, or what you decided. All of that intelligence, yet it fails the baseline requirement of an actual mind: **continuity**.

The naive fix is a bigger context window. That is not memory — it is an expensive scratchpad. As input length grows, cost rises linearly, latency rises super-linearly, and accuracy *drops* (the "lost in the middle" effect). Long context is a tax with diminishing returns.

2.2 The Thesis: Intelligence at Write Time

Standard retrieval-augmented generation (RAG) applies intelligence at **read time**: the agent searches when it needs something. Tottie flips that. It applies intelligence at **write time**: memory curates itself as you live, deciding what matters *before* it is needed. The agent does not manage memory. It just remembers.

- **Observer** runs in parallel and decides what is worth keeping.
- **Scribe** structures, tags, embeds, and *reconciles* what was kept.
- **Retriever** runs on every turn and surfaces what is contextually relevant.
- **Curator** writes the compact *Context Card* the agent actually receives.

2.3 Positioning Against the 2026 Field

The field has converged on a name for this pattern: **learned consolidation**. Live capture $\rightarrow$ LLM extraction of facts and decisions $\rightarrow$ reconciliation (update vs supersede) $\rightarrow$ episodic/semantic split $\rightarrow$ dual retrieval. mem0, Zep/Graphiti, and A-Mem all instantiate it. Published figures show why it matters: mem0 reports **91% lower p95 latency** and **90% fewer tokens** than full-context, for roughly a six-point J-score trade on LOCOMO.

Tottie's two differentiators:

1. **Write-time salience + reconciliation is the product**, not a bolt-on. The system is judged on the decisions it makes at ingestion.
2. **It is transport-agnostic (MCP).** The same brain mounts into a chat client, an IDE, or any MCP host. Memory is infrastructure, not an app feature.

2.4 Why It Matters (the human framing)

The best context is the context you can *automatically* give an agent, and that context is almost always about the person using it. Tottie is one store that knows your life, your preferences, your

decisions, and your infrastructure — so a chat assistant remembers your partner's birthday and a coding agent already knows your stack and database choice before it writes a line.

Part III

Business and Hackathon

Chapter 3

Hackathon Rules, Dates, and Judging

This chapter records the constraints we are optimising against. It is a working extract of the **Meta Global AI Developer Hackathon Participant Guide** so the team never loses sight of the rules.

3.1 Problem Statements

Every project must build toward one of two statements:
- **Build for Your World** — bring Meta's models into a domain you know firsthand.
- **Push the Frontier** — use Meta's models to give a machine a new capability.

Tottie claims *Build for Your World*: an agent memory layer solves a problem the builder knows firsthand (agents that forget), and it generalises to everyone.

3.2 Mandatory Meta Tooling

Projects must incorporate Meta's models: the *Muse Spark* series, *Muse Wonder*, *Muse Image*, *Muse Voice Transcribe*, and *SAM 3.1* via the Meta Model API; run Meta open-weight models (*Muse Glimmer*, *Muse Spark Open*); or build faster with *Muse Code*. Tottie routes its agents (Observer/Scribe/brain) through the Meta Model API, with a local fallback so the demo never dies.

3.3 Dates (all times Eastern)

When	What
Mon Oct 26, 12:00 PM	Kickoff
Tue Oct 27, 12:00–1:00 PM	Agents 101: Build a Multimodal AI Agent
Wed Oct 28, 12:00–1:00 PM	Agents 102: Best Practices for Verifiable Agents
Fri Oct 30, 12:00–1:00 PM	Specialised Intelligence on Fireworks (Muse Open)
Tue Nov 3	Last full day for submission questions
Wed Nov 4, 9:00 PM	**Final office hours; submissions due**
Thu Nov 5 – Sun Nov 8	Judging period
Mon Nov 9, 12:00 PM	Closing ceremony; top 10 announced; winners revealed
Mon Nov 16	Community vote winner announced

Building period: **Oct 27 – Nov 4**. Tottie's initial commit is dated **2026-10-27**.

3.4 Hard Rules

- **Open source:** repositories must be public.

- **Team size:** max four; solo allowed.
- **Demo requirements:** the demo must only highlight features, code, and functionality *built during the Hackathon*. Judges must be able to identify the original contributions. Failure to do so can result in immediate disqualification.
- **New work only:** what is presented must have been built during the event. Existing codebases may be built on, but new work must be clearly identified.
- **Banned:** projects that violate policy, or that use code/data/assets you lack the rights to.

3.5 Sample Anti-Projects (things to avoid resembling)

Basic RAG applications; simple image analysers; generic "AI for education" chatbots; AI job screeners; nutrition coaches; personality analysers; sports analysers; anything political or involving war; medical advice.

3.5.1 How Tottie differs from the banned "Basic RAG Application"

Tottie is *not* a read-time RAG app. It **flips RAG to write time**: the value is the ingestion-time curation decision, and the deliverable is an **MCP server** (infrastructure), not a document-chat UI. This distinction is stated explicitly on Slide 2 of the demo deck.

3.6 Judging Rubric (equally weighted, 0–10)

Category	What it asks
Impact	Real-world potential; who benefits; could people use it; does it fit a statement.
Use of Meta AI Tooling	How creatively Meta's tooling is used — beyond a basic integration.
Technical Depth & Execution	Is the engineering sound and thoughtfully refined; real craft.
Demo	Is it working, impressive, live-safe, and genuinely cool to watch.

3.7 Prizes We Target

Indie Developer track ($400k pool: $150k / $100k / $75k / $50k / $25k), plus bonus awards: **Best Agentic Use-Case** ($5k), **Best Build on Meta Open-Weight Models** ($10k), **Most Viral Project** ($20k), **Community Vote** ($10k).

Chapter 4

Hackathon Compliance Ledger

4.1 Statement of Original Contribution

Every file, commit, and line of code in this repository was authored between **2026-10-27** and **2026-11-04** — the Hackathon building window. No product code pre-existed the event. Where this project reuses architecture, it reuses *patterns* from the author's own repositories (see the Reuse Map), never third-party code or assets the author does not own. This repository is public, satisfying the open-source rule.

4.2 Feature → Commit Ledger

This table is updated at every phase gate. Judges can verify each row against `git log`.

Feature / Component	Commit	Date
Repo scaffold, env, orch config, docs skeleton	`chore: scaffold Tottie repo`	2026-10-27
LaTeX documentation set (`docs/master.pdf`)	`docs: architecture, flows, roadmap, todo`	2026-10-27
Beamer demo deck (`presentation.pdf`)	`docs: demo deck v1`	2026-10-27
Memory engine skeleton (schema, providers, minds)	`feat(core): memory engine skeleton`	2026-10-28
MCP server surface	`feat(mcp): server surface`	2026-10-29
End-to-end memory round-trip	`feat(core): memory round-trip`	2026-10-30
Tottie UI + live Memory Panel + ablation	`feat(ui): Tottie chat + memory panel`	2026-10-31
Reconcile, tag rooms, identity pinning, Lessons	`feat(core): reconcile + tags + lessons`	2026-11-01
MCP-in-IDE integration + eval harness	`feat: mcp-in-agent integration`	2026-11-02
Demo video, README, compliance ledger	`docs: demo video + compliance`	2026-11-03

4.3 Reuse Disclosure

Tottie reuses *architectural patterns* from the author's own repositories. It copies no files. See the Reuse Map in the System Architecture part.

Part IV

System Architecture

Chapter 5

System Architecture

5.1 Overview

Tottie is a memory layer that any agent can mount. The **core engine** owns the memory lifecycle; the **MCP server** is the transport that exposes it; the **Tottie UI** and any **IDE coding agent** are consumers that share the same store.

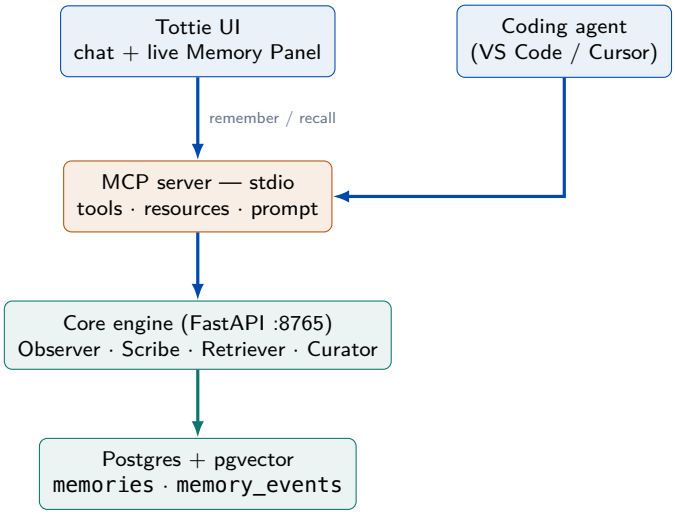


Figure 5.1: Tottie: one brain, three bodies. The UI and any IDE agent mount the same MCP server and share one memory store.

5.2 The Four Minds

Each component has exactly one job. The primary agent (the "present thought") is untouched — it simply receives better context and can *probe*.

Mind	Job	Output
Observer	Runs *in parallel* on every turn; decides "is this worth remembering?"	JSON verdict (keep, salience, reason, candidates)
Scribe	Structure, tag, embed, **reconcile**, route to a tier	rows + a `memory_event`
Retriever	Runs on *every* request; hybrid search across tiers	ranked hits

Curator	Assemble the compact **Context Card** the agent receives	card text + pointers

5.3 The Tier Model

- **Identity** — persona and permanent constraints. Pinned *raw* into the card, zero search overhead. (Example: a partner's birthday.)
- **Episodic** — chronological decisions, state changes, and history.
- **Knowledge** — ingested documents, files, and references (semantic facts).
- **Procedural ("Lessons")** — mistake → problem+solution pairs, surfaced instantly on recurrence ("autosave").

5.4 Write Path (summary)

Observe → embed → reconcile → route tier → persist and emit an event. The live Memory Panel renders each step. Full granular flow in *Process Flows: Write Path*.

5.5 Recall Path (summary)

Embed the utterance → always include the Identity slice → hybrid (vector ∪ full-text) with Reciprocal Rank Fusion → procedural slice → Curator builds the Context Card → emit a recall event. **Dual-path recall:** the card is ambient (auto-injected), the probe is deliberate (on demand).

5.6 Reuse Map (patterns, not files)

Pattern reused	Source (author's own repo)
Embedding load (FastEmbed bge-small)	`ask.nkngo/processor/utils/embeddings.py`
pgvector + HNSW + RRF hybrid search	`ask.nkngo/processor/routers/retrieval.py`
Incremental write + SHA-256 deltas	`ask.nkngo/processor/routers/sync.py`
Relevance floors / abstention	`ask.nkngo/processor/utils/grounding.py`
MCP stdio server scaffold	`Pattern/pattern-vscode-extension/src/mcp-server`
Beamer deck preamble	`Writer/Masters/presentation/.../presentation_*.`
docs/`master.tex` preamble	`Pattern/docs/master.tex`

Chapter 6

Data Model and Storage

6.1 Storage Decision (documented exception)

Local development across the NKNGO ecosystem uses SQLite. Tottie is an explicit, reasoned exception: **semantic memory requires vector search**, and the ecosystem's canonical RAG engine (`ask.nkngo`) and Pattern already run **Postgres + pgvector** locally. Tottie therefore uses a local `tottie` database on `127.0.0.1:5432`. This is a deliberate deviation, recorded here per the workspace's non-destructive documentation standard.

6.2 Schema

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE memories (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  owner       text NOT NULL,                  -- user / tenant scope
  tier        text NOT NULL CHECK (tier IN
  ↪  ('identity','episodic','knowledge','procedural')),
  kind        text NOT NULL,                  -- fact | preference | decision
  ↪  | event | doc | lesson
  subject     text,                           -- e.g. 'girlfriend',
  ↪  'server_c', 'tottie'
  predicate   text,                           -- e.g. 'birthday', 'uses_db',
  ↪  'fix'
  object      text,                           -- e.g. '2026-11-07', 'pgvector'
  content     text NOT NULL,                  -- human-readable memory
  summary     text,                           -- short form for the Context
  ↪  Card
  embedding   vector(384),                    -- bge-small-en-v1.5
  tags        text[] NOT NULL DEFAULT '{}',   -- the "rooms": #person/frans,
  ↪  #project/tottie
  salience    real NOT NULL DEFAULT 0.5,      -- write-time importance 0..1
  confidence  real NOT NULL DEFAULT 1.0,
  source      jsonb,                          -- {session_id, turn, doc, url,
  ↪  msg_id}
  supersedes  uuid REFERENCES memories(id),   -- temporal chain
  valid_from  timestamptz NOT NULL DEFAULT now(),
  valid_to    timestamptz,                    -- NULL = currently true
  hash        text,                           -- dedupe / content hash
  created_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE memory_events (                  -- the X-ray feed + the eval
↪  ledger
```

```
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  ts               timestamptz NOT NULL DEFAULT now(),
  session_id       text,
  turn             int,
  utterance        text,
  observer_verdict jsonb,                    -- {keep, salience, reason,
   ↪  candidates}
  scribe_action    jsonb,                    -- {op, tier, kind, tags,
   ↪  memory_id}
  latency_ms       int,
  tokens           int
);
```

6.3 Indexes

```
CREATE INDEX ON memories USING hnsw (embedding vector_cosine_ops);   --
 ↪  vector recall
CREATE INDEX ON memories USING gin (tags);                            --
 ↪  room/tag filter
CREATE INDEX ON memories USING gin (to_tsvector('english', content)); --
 ↪  full-text (lexical)
CREATE INDEX ON memories (owner, tier) WHERE valid_to IS NULL;        --
 ↪  live-slice lookup
```

6.4 Temporal Correctness

Every fact carries a validity window. When a new fact contradicts an old one, the Scribe sets the old row's `valid_to = now()` and inserts the new row with `supersedes = old.id`. Retrieval filters `valid_to IS NULL`, so **stale facts are never returned** and the agent never has to reconcile in-prompt. This mirrors the temporal-graph behaviour of Zep/Graphiti.

6.5 Owner Scoping

All reads and writes are scoped by `owner`. The demo runs a single owner (`owner_local`), but the schema is multi-tenant from day one: identity, recall, probe, and forget all take an owner key, so the same engine can serve many users without cross-contamination.

6.6 Embeddings

Real embeddings use FastEmbed `BAAI/bge-small-en-v1.5` (384-d), loaded from the local model cache (`../../local LLMS`). A clearly-labelled `TOTTIE_EMBEDDER=hash` dev fallback exists *only* for offline smoke tests and is never presented as production behaviour.

Chapter 7

Model Providers and the Intelligence Layer

7.1 Provider Abstraction

All model calls go through one interface, so the demo can switch models without code changes:

```
class LLMProvider:
    async def complete(self, messages, json_schema=None) -> str: ...
```

- **MetaProvider** — the Meta Model API (Muse Spark series). Configured by `META_API_KEY`, `META_BASE_URL`, `META_MODEL`.
- **LocalProvider** — an OpenAI-compatible local server (Ollama `:11434` or llama.cpp `:8080`), configured by `LOCAL_LLM_URL`, `LOCAL_LLM_MODEL`.
- **resolve_provider()** — uses Meta when a key is present and the endpoint is healthy, otherwise falls back to local. **The demo never dies on a provider outage.**

7.2 Component Model Choices

Mind	Model class	Rationale
Observer	small / fast	high-frequency; a cheap classifier wins
Scribe	small / medium	structured extraction + reconcile; JSON output
Retriever	embeddings only	no LLM in the hot path — bge-small + RRF
Curator	rules (+ optional small)	deterministic card assembly within a token budget
Brain (the agent)	strongest available	the "present thought"; quality matters here

7.3 Cost Control

Tottie's whole argument is that write-time curation is cheaper than stuffing context. The eval harness measures **tokens per query** with memory ON vs OFF to make that claim falsifiable.

7.4 Optional: MCP Sampling (flagged, off by default)

MCP sampling (`ctx.session.create_message`) would let the Observer/Scribe curate using the *host's* model, with **no server-side API key** — an elegant "surprised even us" capability. Two caveats: it is **deprecated in the 2026-07-28 specification** (SEP-2577) in favour of Multi

Round-Trip Requests, and many hosts do not declare the capability. It is therefore **flag-gated** (`TOTTIE_USE_SAMPLING=0` by default) and never the primary path.

Chapter 8

MCP Surface

Tottie ships as an MCP server over **stdio** (Python SDK, `mcp<2` with `FastMCP`). The whole point is that the same memory mounts into any host: the Tottie chat, a VS Code / Cursor agent, anything that speaks MCP.

8.1 Tools

Tool	Signature	Purpose
`memory.remember`	`(text)`	Let the Scribe decide what to keep
`memory.write`	`(tier, content, tags)`	Explicit write to a named tier
`memory.recall`	`(query, tiers?, k?)`	Deliberate hybrid search
`memory.probe`	`(tag|id)`	Walk into a "room" and read its memories
`memory.identity`	`()`	Return the pinned identity facts
`memory.forget`	`(id)`	Right-to-be-forgotten

8.2 Resources

- `memory://identity` — the pinned persona/constraints.
- `memory://tag/{tag}` — the contents of a room, readable without a tool call.

8.3 Prompt

`tottie_context_card` — a reusable system-prompt fragment that teaches a host how to use the memory correctly (when to trust the card, when to probe, when to abstain).

8.4 Dual-Path Recall (the design answer)

A common question is: "does the retriever hand context to the agent, or does the agent probe for it?" Tottie does **both, deliberately**:

1. **Ambient path** — the Curator injects the Context Card automatically on every turn, so the agent "just knows" without issuing a query.
2. **Deliberate path** — the card carries **pointers** (tags/rooms); when the agent needs depth it calls `memory.probe(tag)` and reads that room.

Chapter 9

Tottie UI and the Live Memory Panel

9.1 Layout

A single screen, 16:9-safe for screen-capture: chat on the left, the live Memory Panel on the right.

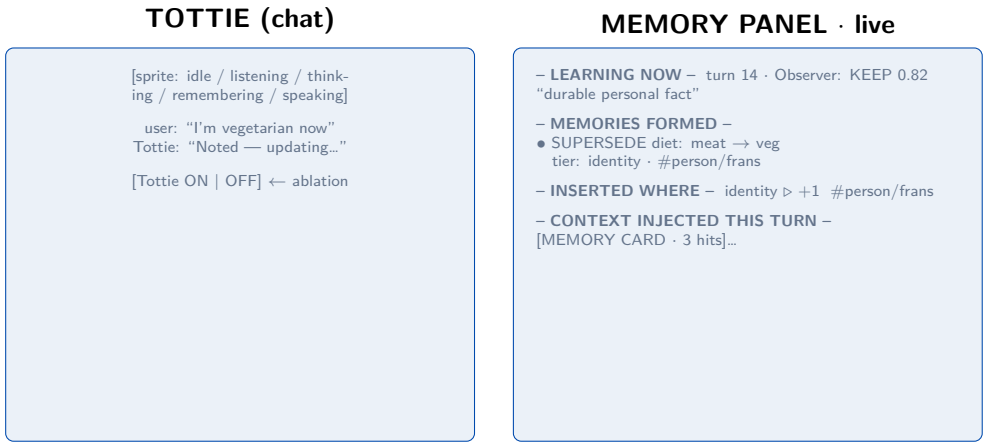


Figure 9.1: The Memory Panel is the hero; the sprite is the reacting character.

9.2 The Panel Is a Live Stream of `memory_events`

The UI subscribes to `GET /events?session_id=` (Server-Sent Events). Each event card shows the four things that matter, in order:

1. **What it is learning now** — the turn plus the Observer verdict (KEEP/DISCARD, salience, reason).
2. **What memories it is forming** — the Scribe operation badge (`NEW` / `UPDATE` / **`SUPERSEDE`** / `SKIP`) and the memory text (superseded ones struck through).
3. **Where it inserted them** — the tier chip and the tag/room chips, plus a live **room map** that grows as new tags appear.
4. **What it injected into context** — the literal **Context Card** the Curator built for that turn. This is what makes the invisible write-time decision visible on camera.

9.3 Tottie Avatar — Drop-In Sprite (frame sets per state)

- **Built here:** a `TottieFace` component plus a state machine with five states — `idle`, `listening`, `thinking`, `remembering`, `speaking` — driven by app events (streaming tokens → `speaking`; a write event → `remembering`; a recall → `remembering`).

- **Provided by the Architect:** frame sets in `ui/public/tottie/<state>/`. Drop PNGs; **no code change required**.
- **Contract:** `TottieFace` reads `{state, framesDir, fps}`. A placeholder frame set ships so the app runs before the final art lands.
- **No TTS / no lip-sync.** The sprite reacts to memory events; that is the whole job.

9.4 Ablation Toggle

A single **Tottie ON/OFF** switch. OFF = no card injected and no writes (raw model). ON = full memory. Used live in the demo to prove that a correct answer *depended on* memory rather than on the base model.

Part V

Process Flows

Chapter 10

Process Flow: The Write Path

Every step below is what the **Memory Panel** renders in its "LEARNING NOW" block. This is the flow that proves the thesis: intelligence applied at write time.

10.1 The User Says Something Durable

Trigger: In the Tottie chat the user types *"my girlfriend's birthday is on the 7th of November"* and presses enter. (Equivalently: an IDE agent turn completes and the host forwards it.)

Action / UI State: The Tottie sprite leaves `idle`, enters `listening` for the duration of the send, then `thinking`. The Memory Panel prepends a new turn card with an empty "LEARNING NOW" row and a subtle processing shimmer.

System Logic: The host issues `POST /write` with `{text, session_id, turn, owner}`. The core hands the turn to the **Observer**, which runs *in parallel* with the agent's own response generation.

Data Flow & Mutators: The turn is not yet stored as memory; it exists only as an inbound request and a `memory_event` row in `queued` state.

10.2 The Observer Decides

Trigger: The Observer receives the turn tuple.

Action / UI State: The "LEARNING NOW" row fills in: `Observer:` KEEP (or DISCARD), a salience score (e.g. 0.82), and a short reason ("durable personal fact").

System Logic: The Observer calls the resolved LLM provider with a strict JSON contract and returns a verdict:

```
{ "keep": true, "salience": 0.82, "reason": "durable personal fact",
  "candidates": [ { "tier":"identity","kind":"fact","subject":"girlfriend",
    "predicate":"birthday","object":"2026-11-07",
    "content":"Girlfriend's birthday is 7 November",
    "tags":["#person/girlfriend","#topic/birthday"] } ] }
```

Data Flow & Mutators: The verdict JSON is written into the `observer_verdict` column of the `memory_event` and pushed to the Panel over SSE. If `keep` is false, the flow ends here and the Panel shows a greyed DISCARD chip.

10.3 The Scribe Structures and Embeds

Trigger: A non-empty candidate list from the Observer.

Action / UI State: The Panel's "MEMORIES FORMED" block gains a row per candidate, showing the operation badge and the memory text.

System Logic: For each candidate the Scribe (1) composes the human-readable memory, (2) computes its embedding with `bge-small-en-v1.5`, (3) assigns tags, and (4) hands the candidate to the reconciler (next chapter).

Data Flow & Mutators: An embedding vector (384 floats) is computed in memory; the candidate is held for the reconcile step.

10.4 Reconcile, Route, and Persist

Trigger: An embedded candidate.

Action / UI State: The Panel shows the operation badge (`NEW` / `UPDATE` / `SUPERSEDE` / `SKIP`), the tier chip (e.g. `identity`), and the room chips (e.g. `#person/girlfriend`).

System Logic: See *Process Flow: Reconcile and Temporal Correctness* for the full decision table. The routed tier is recorded, and the live row is inserted with `valid_from = now()`.

Data Flow & Mutators: `INSERT INTO memories (...)`; the `memory_event.scribe_action` column is filled; the Panel's "INSERTED WHERE" block and the **room map** update; the new memory becomes immediately retrievable.

10.5 Why This Order Matters

Because the decision is made *at write time*, no read-time query is ever needed to "know" this fact. The next time the user asks anything related, the Retriever simply finds it — the agent never had to remember to save it.

Chapter 11

Process Flow: The Recall Path

This is the "middle part": what happens between the user's utterance and the context the agent receives. Tottie answers it with the **Context Card** (ambient) plus the **probe** (deliberate).

11.1 A New Turn Arrives

Trigger: The user sends *"when is her birthday again?"* in the chat, or a fresh session opens and the agent needs context about the user.

Action / UI State: The sprite enters `thinking`; the Panel prepares a "CONTEXT INJECTED THIS TURN" block.

System Logic: The host calls `POST /recall` (or `memory.recall` over MCP).

Data Flow & Mutators: The utterance (and optionally the last assistant turn) is embedded with `bge-small-en-v1.5`.

11.2 The Identity Slice Is Always Included

Trigger: Any recall. **Action / UI State:** The Panel always shows the pinned identity line at the top of the card. **System Logic:** `SELECT ... WHERE tier='identity' AND valid_to IS NULL`. This is unconditional and cheap — identity has *zero search overhead*. **Data Flow & Mutators:** The identity facts are collected for the card header.

11.3 Hybrid Search

Trigger: The utterance embedding plus the owner scope. **Action / UI State:** Nothing yet — the Panel shows a "searching" shimmer. **System Logic:**

1. **Vector** leg: HNSW top-50 by cosine.
2. **Lexical** leg: full-text top-50.
3. **Filter**: `owner = ?` AND `valid_to IS NULL` (and any tag filter).
4. **Fusion**: Reciprocal Rank Fusion, $k = 60$, to top-8.
5. **Re-rank**: weight by salience · recency · confidence.

Data Flow & Mutators: A ranked hit list of `(id, tier, summary, tags, score)`.

11.4 The Procedural Slice

Trigger: The utterance resembles a previously-solved problem (an error signature or a similar embedding). **System Logic:** Match `kind='lesson'` rows; if a match clears the threshold, the solution is surfaced *instantly* — the "autosave" behaviour. **Data Flow & Mutators:** Matched lessons are appended to the card.

11.5 The Curator Builds the Context Card

Trigger: Identity slice + ranked hits + procedural matches. **Action / UI State:** The Panel's "CONTEXT INJECTED THIS TURN" block shows the exact card. **System Logic:** The Curator assembles a compact card (<400 tokens) with one line per hit, its tier, its source, and its tags, plus a "probe deeper" pointer line:

```
[MEMORY CARD - 3 hits]
  IDENTITY (pinned)  user=Frans - tz=CAT - girlfriend birthday = Nov 7
  EPISODIC 2026-10-29  decided: pgvector not Qdrant for Tottie    #tottie
  ↪  #decision
  KNOWLEDGE (doc: telecom-arch.pdf p.7)  Server C sits behind A/B  #telecom
  ↪  #infra
  -> probe deeper: memory.probe("#telecom") | memory.recall("server C
  ↪  topology")
```

Data Flow & Mutators: The card text is returned to the host and injected into the prompt space; a recall `memory_event` is written and streamed to the Panel.

11.6 The Deliberate Probe

Trigger: The agent decides it needs more depth than the card provides. **Action / UI State:** The Panel opens the targeted room and highlights the returned memories. **System Logic:** `memory.probe(tag)` (or `probe(id)`) returns the room's live memories. **Data Flow & Mutators:** A read-only fetch scoped by owner and `valid_to IS NULL`.

11.7 Ablation

Trigger: The Tottie ON/OFF toggle is set to OFF. **System Logic:** The recall step is skipped entirely — no identity slice, no hits, no card. The base model answers with no memory. This is the control condition used in the demo and the eval harness.

Chapter 12

Process Flow: Reconcile and Temporal Correctness

Reconciliation is what separates a real memory from a pile of embeddings. It is the moment a new fact meets an old one.

12.1 The Decision Table

Trigger: An embedded candidate arrives from the Scribe.

System Logic: The reconciler searches for an existing memory for the same owner matching `(subject, predicate)` *or* a cosine similarity above `TOTTIE_RECONCILE_COSINE` (default 0.86) within the same tags, then decides:

Operation	When	What happens
`NEW`	No prior memory matches	Insert with `valid_from = now()`
`SKIP`	A matching memory already holds the same value	Bump confidence / salience; no new row
`UPDATE`	A matching memory is enriched but not contradicted	Append detail; refresh summary
`SUPERSEDE`	A matching memory holds a *different* value	Set old `valid_to = now()`; insert new with `supersedes = old.id`

Data Flow & Mutators: Any insert or update is written to `memories`; the decision is written to `memory_event.scribe_action` and streamed to the Panel, where a superseded memory is rendered with a strikethrough.

12.2 Worked Example: "I'm vegetarian now"

Trigger: The user says *"actually, I'm vegetarian now"*.

Action / UI State: The Panel's "MEMORIES FORMED" block shows a red-ish SUPERSEDE badge, the old value struck through ("diet: meat-eater"), and the new value ("diet: vegetarian").

System Logic:

1. The Observer flags a durable preference change (salience high; `kind=preference`).
2. The Scribe embeds the candidate and finds the live fact (`subject=user, predicate=diet, object=meat-eater`).
3. The values differ, so the operation is SUPERSEDE.
4. The old row's `valid_to` is set to `now()`; the new row is inserted with `valid_from = now()` and `supersedes = old.id`.

Data Flow & Mutators: Two row mutations and one event; retrieval now returns only "vegetarian" because it filters `valid_to IS NULL`. The agent never has to reconcile in-prompt, and it can never quote a stale fact.

12.3 Why This Is Hard and Why It Wins

Plain vector recall has *no update semantics*: both the old and the new fact retrieve, and the model is left to guess which is current. Tottie resolves the conflict at write time, once, and records the temporal chain. This is precisely the capability that LongMemEval's *knowledge update* category and MemoryAgentBench's *selective forgetting / conflict resolution* category probe — and it is a headline slide in the demo.

Chapter 13

Process Flow: The Demo Click-Path

This is the exact physical sequence the presenter performs on stage and that the screen-capture video follows. Each beat is written so a second operator could run the demo cold.

13.1 Beat 1 — Cold Open (Amnesia)

Trigger: A plain agent window with no memory. **Action / UI State:** The agent greets the user and does not know their name. **System Logic:** No Tottie in the loop. **Data Flow & Mutators:** None. **On screen:** caption “A year of conversations.”

13.2 Beat 2 — Tottie Wakes (Identity Pinning)

Trigger: Open the Tottie chat. **Action / UI State:** The sprite enters `idle`; the Panel immediately shows the pinned Identity line: `girlfriend birthday = Nov 7`. **System Logic:** The Panel performs a recall; the Identity slice is returned with zero search. **Data Flow & Mutators:** A read of `tier='identity'` rows. **On screen:** caption “It just…knows.”

13.3 Beat 3 — Write-Time Magic (Supersede)

Trigger: Type *“I’m vegetarian now”*. **Action / UI State:** The Panel shows `Observer:` KEEP, then a SUPERSEDE badge with the old value struck through; the room map gains a tag. **System Logic:** The full write path plus reconcile (see the Write Path and Reconcile flows). **Data Flow & Mutators:** Old row closed; new row inserted; event streamed.

13.4 Beat 4 — Three Bodies (Cross-Agent Recall)

Trigger: Switch to VS Code; ask the coding agent about the project’s stack. **Action / UI State:** The coding agent answers with the database and architecture decisions it was never told in this session — because they were written in the Tottie chat. **System Logic:** Same MCP server, same store, different host. **Data Flow & Mutators:** A `memory.recall` over MCP from the IDE. **On screen:** the caption “one brain, three bodies.”

13.5 Beat 5 — Autosave a Mistake

Trigger: The coding agent hits a known error. **Action / UI State:** The Panel shows a NEW lesson being formed (problem → solution). **System Logic:** A `kind=lesson` memory is written under `tier=procedural`. **Data Flow & Mutators:** Insert into `memories` with the error signature for matching.

13.6 Beat 6 — Restart + Ablation

Trigger: Kill the session; reopen; ask a question answered 20 minutes ago. **Action / UI State:** It answers correctly. Flip **Tottie OFF**; the same question now returns "I don't know"; flip **ON**; it answers again. **System Logic:** The OFF state skips the recall path entirely. **Data Flow & Mutators:** With OFF, no card is injected.

13.7 Beat 7 — The Joke and the Pitch

Trigger: Back in the chat. **On screen:** caption *"…everything I was mad at my girlfriend for — and the one thing that matters: her birthday."* Then the universal pitch: autosave for mistakes, a personal AI that knows you, and the company brain (Server C behind Server A and B).

Part VI

Planning

Chapter 14

Roadmap

14.1 Scope Tiers (9 days, solo, alongside work and study)

Tier	Contents
MUST	Core engine · MCP server · Context Card + probe · reconcile/temporal + identity pinning · Tottie UI (chat + sprite state machine + live Memory Panel) · ON/OFF ablation · eval harness · 3-minute video · docs + deck
SHOULD	Coding-agent MCP integration (three bodies) · Autosave/Lessons · tag-rooms map
STRETCH	MCP-sampling curation mode · ingest telecom docs as Knowledge · relation edges (GraphRAG-lite) · X/viral push

14.2 Phase Schedule

Phase	Dates	Deliverable
P0	Oct 27	Scaffold: repo, gitignore, env, orch config, docs skeleton, first commit
P1	Oct 27	Docs: all `docs/**/*.tex` → `master.pdf`
P2	Oct 27–28	Deck: `presentation.tex` → `presentation.pdf`
P3	Oct 28–29	Core: engine + schema + providers + four minds
P4	Oct 29	MCP: server tools/resources wired
P5	Oct 30	End-to-end memory round-trip
P6	Oct 30–31	UI: chat + sprite + live Memory Panel + ablation
P7	Oct 31–Nov 1	Reconcile, tag rooms, identity pinning, Lessons
P8	Nov 1–2	MCP-in-IDE integration + eval harness + tests green
P9	Nov 3	3-minute video, README, recompile docs, compliance ledger
P10	Nov 4	Repo public, video link, credits claimed, submit by 9 PM ET

14.3 Gating

A phase does not begin until the prior phase's **docs** are written and its tests (where applicable) pass. If time runs short, cut in this order: STRETCH → tag-rooms map → autosave. **Never cut the eval harness or the demo video.**

14.4 Mandates Honoured During the Build

Document First → Implement → Test → Conclude; atomic commits after every step; every `.tex` auto-compiled to PDF; env vars in both `orch.config.json` and `.env.example`.

Chapter 15

Master Todo

15.1 Phase 0 — Scaffold

☐ Create repo tree; `git init`; branch `main`
☐ `.gitignore`, `.env.example`, `orch.config.json`, `Makefile`
☐ `README.md`, `GEMINI.md`, `.agents/orch-directive.md`
☐ First commit dated 2026-10-27

15.2 Phase 1 — Documentation

☐ `master.tex` + every part `*.tex`
☐ Compile `master.pdf` (`latexmk -xelatex`); zero undefined refs
☐ `hackathon_rules.tex`, `hackathon_compliance.tex`

15.3 Phase 2 — Deck

☐ `presentation.tex` (Beamer, Madrid/whale, 16:9)
☐ Compile `presentation.pdf` (`pdflatex` $\times 2$)

15.4 Phase 3 — Core Engine

☐ `config.py`, `db.py`, `schema.sql`
☐ `embeddings.py` (bge-small + labelled hash fallback)
☐ `providers/` (base, meta, local, resolve)
☐ `agents/observer.py`, `scribe.py`, `retriever.py`, `curator.py`
☐ `tiers.py`

15.5 Phase 4 — MCP

☐ `mcp/server.py`: 6 tools + 2 resources + `tottie_context_card`
☐ `--list-tools` prints the surface

15.6 Phase 5 — Round-Trip

☐ `POST /write` $\rightarrow$ store $\rightarrow$ `POST /recall` $\rightarrow$ Card
☐ `GET /events` SSE stream for the Panel
☐ Preflight test: write one fact, recall it

15.7 Phase 6 — UI

☐ `TottieFace` (5-state machine + frame-set contract + placeholders)
☐ `ChatStream`, `MemoryPanel`, `RoomMap`, `AblationToggle`
☐ Panel consumes SSE; renders the four blocks

15.8 Phase 7 — Reconcile / Tags / Identity / Lessons

☐ Temporal supersede with `valid_to` / `supersedes`
☐ Hierarchical tags / rooms; identity allowlist
☐ `kind=lesson` autosave + recurrence match

15.9 Phase 8 — Integration + Eval

☐ Mount MCP in VS Code / Cursor (three bodies)
☐ `eval/harness.py` + `scenarios.jsonl` (ON/OFF ablation)
☐ All tests green

15.10 Phase 9 — Film

☐ 3-minute screen-capture video (captions + music)
☐ README final; recompile docs; update compliance ledger

15.11 Phase 10 — Submit

☐ Make repo public
☐ Video link accessible
☐ Claim Meta credits (<Wed Nov 4, 9 PM ET)
☐ Submit via the CV platform

Chapter 16

Demo Script (3 minutes, screen capture + captions + music)

Format: pure screen capture, on-screen captions, a music bed. No talking head. Total runtime **3:00**. Every caption is written to be readable on a phone.

16.1 Shot List

Time	Beat	On screen
0:00	Cold open	Black. Caption: *"A year of conversations."* A plain agent: *"Hi — I don't know you."* Music swells.
0:12	The problem	Restart = amnesia; the agent re-asks your name. Caption: *"Long context isn't memory."*
0:30	The reveal	Sprite wakes; Panel pins Identity `birthday = Nov 7`. Caption: *"It just…knows."*
1:05	Write-time magic	*"I'm vegetarian now"* → Panel: SUPERSEDE (old struck through); a new fact → Observer verdict + Scribe NEW live; room map grows.
1:35	Three bodies	Cut to VS Code: the coding agent already knows the stack; it hits a bug; **autosave** a lesson; a new session recalls the fix.
2:05	Restart + ablation	Kill the session, reopen, ask an old question → knows. Flip **Tottie OFF** → *"I don't know."* ON → knows.
2:40	Joke + pitch	Caption: *"…everything I was mad at my girlfriend for — and the one thing that matters: her birthday."* → the aspirational line + repo link.

16.2 Narration Captions (verbatim)

1. "A year of conversations."
2. "It still doesn't know your name."
3. "Long context isn't memory — it's a tax."
4. "Tottie applies intelligence at write time."
5. "It decides what matters before you ever need it."
6. "One brain. Three bodies. Your chat, your code, the panel."
7. "Turn it off, and it forgets. Turn it on, and it knows."
8. "So — I used it to remember everything I was mad at my girlfriend for. And the one thing that actually matters: her birthday."

16.3 Music and Pacing

Cinematic, rising, aspirational under the reveal; a beat drop on the ablation flip; a warm settle under the joke. Cuts land on caption boundaries. Keep on-screen text large; assume a phone screen.

Chapter 17

Plan Audit (Numbered Outline and Method)

This chapter makes the plan *auditable*. Every part of it carries a stable ID so a second reviewer (human or agent) can judge it section by section without asking for context.

17.1 Audit Method

For each numbered item we record one verdict against six fixed axes:

Axis	Question
Feasibility	Can one person build this Oct 27–Nov 4 alongside work and study?
Dependency	What must exist first? Does it already exist (Reuse Map)?
Risk	What breaks it — live on stage, or under judge scrutiny?
Rubric	Which judging column it serves (Impact / Meta tooling / Technical depth / Demo).
Cost	Effort: S (hours) · M (a day) · L (multi-day).
Verdict	Keep · Improve · Cut · Add · Defer.

17.2 Independent-Review Questions (for third-party judges)

1. Does it satisfy the problem statement (*Build for Your World* / *Push the Frontier*)?
2. Is Meta's tooling central, not decorative?
3. Is the engineering sound and thought through?
4. Is the demo compelling *and live-safe*?
5. Is it clearly new work, distinguishable from reuse?
6. Is anything here a banned anti-project?
7. Are the claims measurable (not vibes)?
8. Is the scope honest for 9 solo days?

17.3 The Numbered Outline

1. **Meta & scope** — event/dates; repo/branch/commit rule; decision log; scope note.
2. **Strategy** — thesis; positioning vs mem0/Zep/Letta; differentiators; problem statement.
3. **Demo design** — three bodies; the money shot; video style.
4. **Architecture** — four minds; tiers; write path; reconcile; tier routing; recall path; Context Card; probe/tags; concurrency.
5. **Data & storage** — `memories`; `memory_events`; indexes; pgvector decision; embeddings; init; owner scoping.

6. **Intelligence layer** — Observer; Scribe; Curator; providers; model choice/cost; sampling.
7. **Interfaces** — HTTP surface; MCP tools; resources; prompt; SSE events; error/abstention.
8. **UI/UX** — layout; panel blocks; sprite state machine; ablation; room map; typography.
9. **Evaluation** — scenario set; ablation method; metrics; harness; rubric mapping.
10. **Execution plan** — MUST/SHOULD/STRETCH; phase schedule; gating; mandates.
11. **Media** — shot list; deck; music/captions; viral plan.
12. **Repo & tooling** — tree; reuse map; orch/env; docs structure; build commands; gitignore/Makefile.
13. **Compliance** — rules mapping; contribution ledger; prize targeting; banned-avoidance.
14. **Risk** — register; live-demo mitigations.
15. **Verification** — test plan; dry-run checklist; orch checks.
16. **Open items** — sprite frames; Meta keys; sampling; name/branch.
17. **Appendices** — rules extract; glossary; sources.

17.4 Audit Ledger

The per-item verdicts are maintained alongside this specification. Items are resolved to `Keep`/`Improve` before the spec is frozen for build.